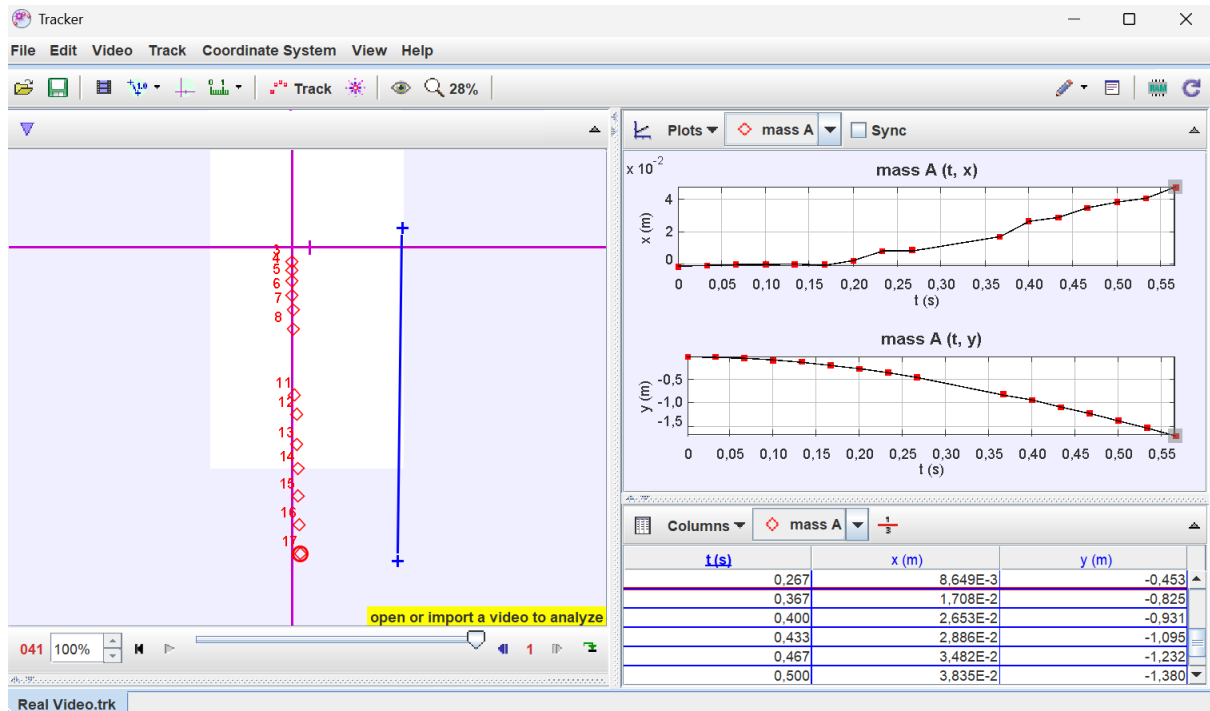


Part 1:

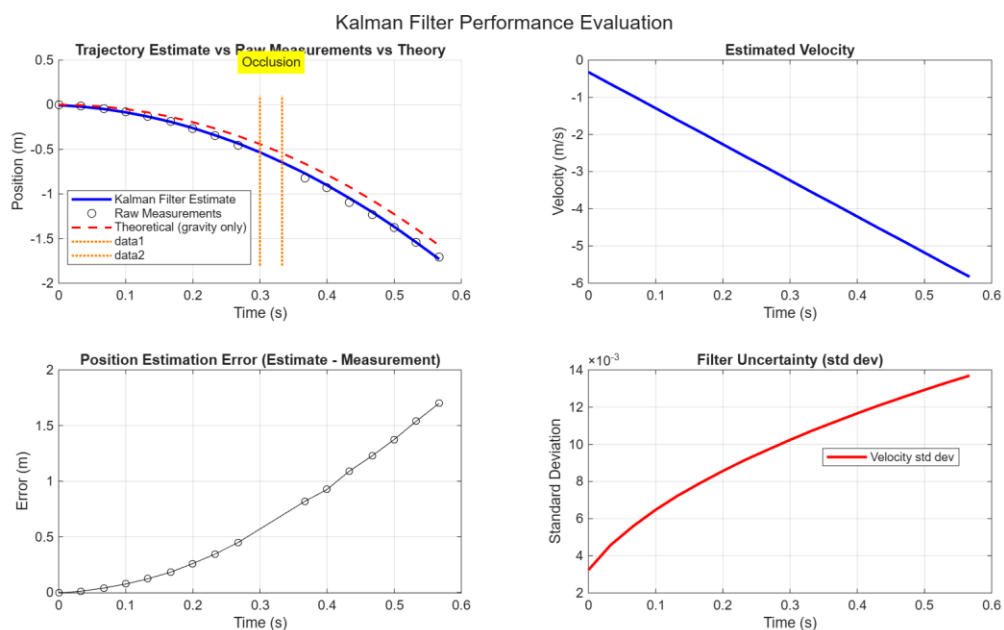
FPS = 120

Drop height = 1.705 m

Duration of occlusion = 0.567s



Part 4: Plots and analysis



Top Left Trajectory Estimate vs Raw Measurements vs Theory

The Kalman filter (blue) acts as an optimal smoother between the noisy raw measurements (circles) and pure theoretical gravity (red dashed). The filter doesn't just follow measurements blindly it weighs them against the dynamic model. During the occlusion window ($t = 0.3\text{--}0.333\text{s}$, marked in yellow), no measurements are available, so the filter relies purely on the physics model ($Bu(k)$ term) to propagate the state forward. The fact that the blue line stays close to theory during occlusion confirms the model is working correctly.

Top Right — Estimated Velocity

Velocity is never directly measured (since $H = [1\ 0]$), yet the filter estimates it purely by inferring from position changes over time. The smooth linear decrease confirms the filter correctly captures constant gravitational acceleration of -9.81 m/s^2 . This plot is important because:

- It validates the second state is being tracked correctly
- It confirms no measurement of velocity is needed — the Kalman filter infers it
- Any sudden jumps would indicate filter instability

Bottom Left — Position Estimation Error

The growing error ($0 \rightarrow \sim 1.7\text{m}$) is expected and physically meaningful. There are two reasons it grows:

- Occlusion — during missing measurements, only the model propagates the state, and small model errors accumulate
- K remains small — because $P(0) = 0$ gave perfect initial confidence, the filter trusts measurements very little, so corrections are minimal

This plot is important because it tells you when the filter starts becoming unreliable and whether you need more frequent measurements or a larger Q to force the filter to trust measurements more.

Bottom Right — Filter Uncertainty (Std Dev)

- Position std dev stays near zero — the filter is very confident in position because it's directly measured
- Velocity std dev grows over time because velocity is never directly observed, only inferred

This is critically important because:

- During occlusion, no measurements correct the velocity estimate, so uncertainty grows
- The curved growth flattening out shows the filter reaching a steady-state uncertainty balance between Q (process noise) and R (measurement noise)
- If velocity uncertainty grows unbounded, it signals your Q is too small relative to real-world disturbances

Overall Takeaway

The filter is behaving correctly. The growing error is not a failure — it reflects the fundamental trade-off: small K (trusting the model) gives smooth estimates but slow correction. If you wanted faster error correction, you could increase $P(0)$ or Q to make the filter trust measurements more aggressively.

Part 5:

5.1 Measurement Noise

R is the measurement noise covariance increasing it tells the filter to trust measurements less, trust the model more.

Below is the performance of the filter before and after multiplying by 10.

=== Performance Statistics Before ===

Mean position error: 0.014035 m

RMS position error: 0.793381 m

Max position error: 1.730591 m

Occlusion duration: 0.066 s

== Performance Statistics After ==

Mean position error: 0.014097 m

RMS position error: 0.793327 m

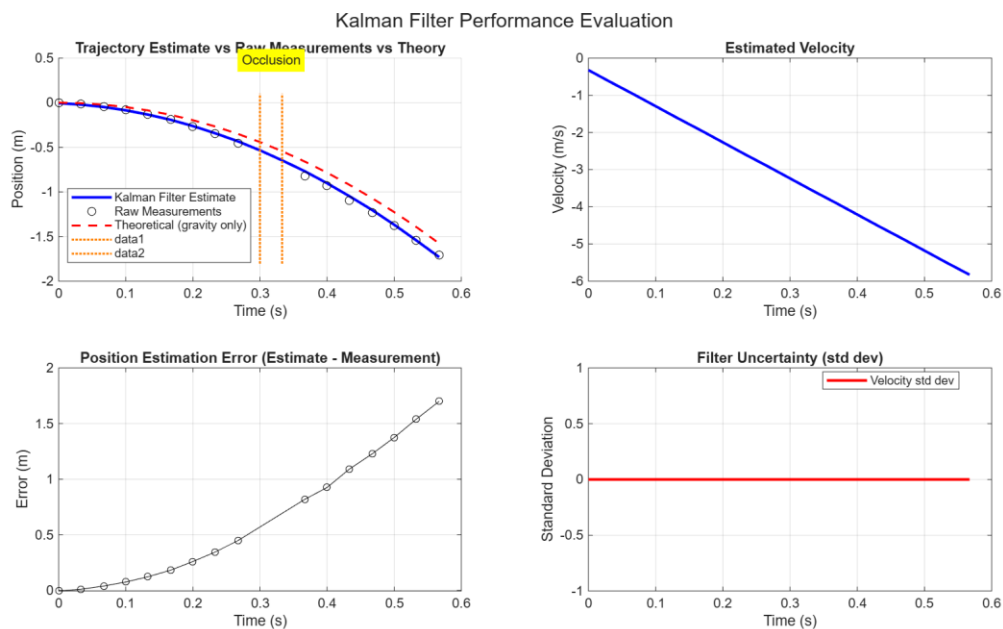
Max position error: 1.730386 m

Occlusion duration: 0.066 s

By looking at these results multiplying R by 10 only did a slit change.

5.2 Process Noise

Before: Q is the process noise covariance setting it to zero tells the filter "the model is perfect, there is no uncertainty in the physics".



After: The error plot looks similar to before, but the uncertainty plot shows zero meaning the filter is making large errors while being completely confident it isn't. This is the exact definition of a diverging/overconfident filter and is why $Q = 0$ should never be used in practice.

5.4 Initial Conditions

Small P_0 tells the filter "I am very confident in my initial state" even though that state is wrong. If you set P_0 small with a wrong state, the filter is confidently wrong and stays wrong. If you set P_0 large with a wrong state, the filter admits uncertainty and fixes itself.

5.5 Wrong Process Model

When removing G , the filter now believes the object moves at constant velocity it has no knowledge that gravity is accelerating it downward.

Part 6: Conceptual Questions

Question 1:

The Kalman gain is essentially the filter's decision-maker it determines how much to trust a new measurement versus how much to trust the model's prediction. Think of it as a confidence dial: when the gain is high, the filter leans toward the measurement to correct the state; when it is low, the filter leans toward what the physics model predicted. In your falling object problem, because P_0 was set to zero, the gain started extremely small, meaning the filter was so confident in its initial state that measurements had almost no influence. Physically, the gain answers the question: "given how uncertain my model is compared to how noisy my sensor is, how much should I adjust my estimate right now?"

Question 2:

The predict and update steps are separate because they represent two fundamentally different sources of information that arrive at different moments in time. The prediction step uses the physics model to project the state forward in time it answers "where should the object be now based on what I knew before?" The update step then incorporates the sensor measurement to correct that projection it answers "now that I have a reading, how should I revise my estimate?" Collapsing them into one step would destroy this temporal structure, because you would lose the ability to propagate uncertainty correctly between measurements, which is especially critical during occlusion when no measurement arrives and the filter must rely purely on prediction for multiple steps.

Question 3:

The student is correct in a narrow but meaningful sense: the updated state is indeed a blend of the prediction and the measurement, weighted by the Kalman gain, which itself is derived from the ratio of model uncertainty to measurement uncertainty. So yes, if the model is very uncertain, the measurement gets a higher weight, and if the sensor is very noisy, the prediction gets a higher weight this is the spirit of a weighted average.

However calling it simply a weighted average is an oversimplification because it misses the fact that the weights themselves are not fixed they are dynamically computed every iteration based on the evolving covariance matrix P , which tracks how uncertainty propagates and grows through the system. Furthermore, the Kalman filter is optimal in the least-squares sense under Gaussian noise assumptions, which goes far beyond what a simple weighted average implies.

Question 4:

Without switching to a nonlinear filter like the Extended Kalman Filter, the most practical approach is to increase the process noise covariance Q to account for the fact that your model is imperfect. By inflating Q , you are essentially telling the filter "I know my model does not perfectly describe reality, so treat the unmodelled forces as additional random disturbances." Another option is to augment the state vector with an additional bias state representing the unknown acceleration deviation, allowing the filter to slowly estimate and compensate for the drag effect over time. The trade-off is that increasing Q makes the filter trust measurements more aggressively, which improves tracking of real dynamics but also makes the estimate noisier and more sensitive to measurement outliers you gain robustness to model error at the cost of smoothness.

Question 5:

The state vector should track at minimum position, velocity, and heading, and possibly acceleration if the vehicle dynamics warrant it, since both sensors contribute information about these quantities but at different rates and accuracies. The two sensors with different update rates are handled by running the prediction step at the faster odometer rate (100 Hz) and only performing the GPS update step when a GPS measurement arrives (1 Hz) the filter naturally accommodates this because prediction

and update are decoupled. For Q , you would set it based on expected disturbances like road bumps, steering corrections, and unmodelled dynamics it needs to be large enough that the filter stays responsive over the one-second gaps between GPS fixes, because during that window only the odometer and the model keep the estimate alive. For R , the GPS measurement noise matrix would have large values ($\sim 4 \text{ m}^2$ for the 2 m accuracy) while the odometer R would be very small ($\sim 0.0001 \text{ m}^2$) for short intervals, but critically you would also need to model odometer drift over time, either by increasing its R gradually or by adding a drift bias state otherwise the filter will over-trust the odometer and accumulate large long-term position errors.